

Complete DSA Learning Order

Topic Content Compilation

Section 9 — Heaps / Priority Queues (9.1 – 9.4)

June 30, 2026

Section 9 — Heaps / Priority Queues

9.1 — Heap property — min-heap vs max-heap

1. PLAIN-LANGUAGE GIST

A heap is a tree-shaped structure that enforces one simple rule at every node — in a min-heap, every parent is smaller than (or equal to) its children; in a max-heap, every parent is larger than (or equal to) its children. This weaker, parent-only ordering rule (compared to a BST's full left/right ordering) is exactly what lets a heap answer “what's the smallest/largest element?” instantly, in $O(1)$, while still supporting efficient insertion and removal.

2. REAL-WORLD ANALOGY

Picture a corporate hierarchy where the only rule is “every manager's salary is lower than (or equal to) every one of their direct reports' salaries” (an inverted org chart, for a min-heap). You can immediately point to the very top of the chart and know that's the lowest-paid person in the company — but you have no idea, just from the chart shape, whether one manager's sibling manager's reports earn more or less than this manager's own reports. That sibling-to-sibling comparison is simply never guaranteed.

3. CONNECT TO PRIOR KNOWLEDGE

This directly contrasts with 8.4's BST property — a BST enforces ordering between a node and both of its entire subtrees (left smaller, right larger), while a heap only enforces ordering between a node and its immediate children, a deliberately weaker constraint that trades away BST's “inorder traversal gives sorted order” property in exchange for $O(1)$ access to just the minimum or maximum.

4. CORE MECHANISM (Simple → Intermediate)

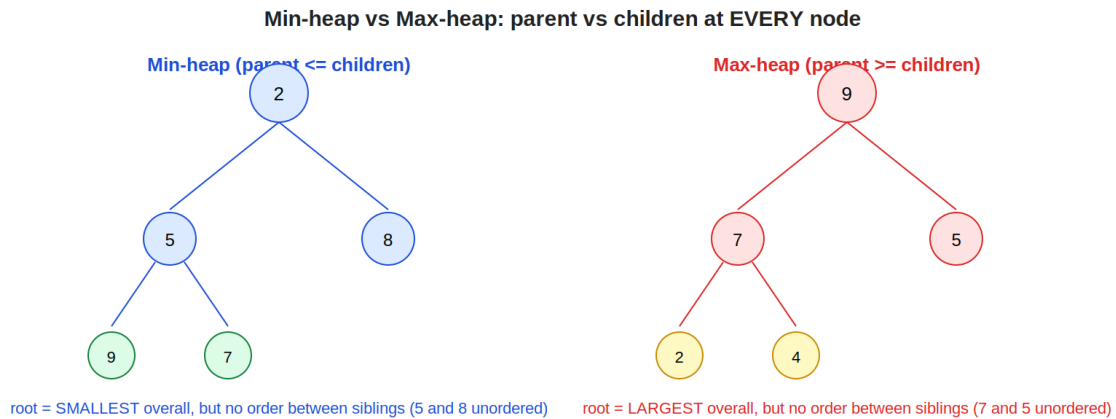
Simple model: A min-heap is a binary tree where every node's value is less than or equal to both its children's values — this guarantees the root is always the overall smallest element. A max-heap is the mirror image: every node's value is greater than or equal to its children, guaranteeing the root is always the overall largest.

Next layer — what the heap property does and doesn't guarantee, and how heaps stay efficient:

- **No ordering between siblings or across subtrees** — in a min-heap, a node's left child could be larger or smaller than its right child; the only guaranteed relationship is parent-to-child, not sibling-to-sibling or across different branches. This is precisely why a heap can't answer “what's the second smallest element?” without extra work, unlike a sorted structure.
- **Heaps are typically implemented as a complete binary tree stored in a plain array** — no explicit node objects or pointers needed. A node at array index i has children at indices $2i+1$ and $2i+2$, and a parent at index $(i-1)/2$ — this compact representation avoids the pointer overhead of an explicit tree structure (3.1's nodes) and keeps the tree always “complete” (filled left to right, no gaps), which is essential for the array-index math to stay valid.
- **Insertion:** add the new element at the end of the array, then “bubble up” — repeatedly swap it with its parent as long as it violates the heap property, which takes at most $O(\log n)$ swaps since the tree's height is $O(\log n)$ (a complete tree is automatically balanced, unlike 8.1's unbalanced BST risk).

- **Extraction** (removing the root): move the last element in the array into the root position, then “bubble down” — repeatedly swap it with whichever child violates the heap property, again $O(\log n)$ swaps in the worst case.
- **Peek** (just looking at the root) is $O(1)$ — no traversal needed at all, since the root is always the array's first element by construction.

5. VISUAL REPRESENTATION



The heap property is ONLY parent-vs-child, at every level -- much weaker than a BST's full ordering, which is exactly why heaps achieve $O(1)$ peek but only $O(\log n)$ extract.

The diagram shows a min-heap (root is the smallest value, 2) and a max-heap (root is the largest, 9) side by side — in both cases, every parent correctly relates to its children, but sibling subtrees (the two children of the root) have no guaranteed relationship to each other, which is the core, easily-missed limitation of the heap property.

6. WHEN TO USE / WHEN NOT TO USE

Use a heap when:

- You repeatedly need the current minimum (or maximum) from a changing collection, with frequent insertions and removals — task scheduling by priority, simulation event queues, or any “always process the most urgent thing next” pattern.
- You're solving a “k largest/smallest” or “k-th largest” style problem (formalized in 9.3) — a heap is the standard tool for maintaining a running top-k set efficiently.

Wrong choice when:

- You need ordering information beyond just the min/max — a heap cannot efficiently answer “what's the median?” or “give me all elements in sorted order” without additional structure (9.4) or essentially dismantling the heap entirely.
- You need fast arbitrary search for a specific value — a heap's weak ordering doesn't support binary-search-style elimination (6.1, 8.4); finding an arbitrary value in a heap is $O(n)$ in the worst case, no better than scanning an unordered array.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — verifying the heap property holds, conceptually, on an array-backed min-heap:

```

bool IsValidMinHeap(int[] heap) {
    for (int i = 0; i < heap.Length; i++) {
        int left = 2 * i + 1, right = 2 * i + 2;
        if (left < heap.Length && heap[i] > heap[left]) return false;
        if (right < heap.Length && heap[i] > heap[right]) return false;
    }
    return true;
}

```

This directly demonstrates the array-index relationship ($2i+1$, $2i+2$ for children) described above, and shows that checking the heap property is just a simple parent-vs-child comparison at every index, nothing more elaborate.

INTERMEDIATE — manual “bubble up” insertion into a min-heap array, illustrating the mechanism behind Enqueue (formalized in 9.2):

```

void BubbleUp(List<int> heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (heap[index] >= heap[parent]) break; // heap property OK
        (heap[index], heap[parent]) = (heap[parent], heap[index]);
        index = parent; // continue from parent's position
    }
}

void Insert(List<int> heap, int value) {
    heap.Add(value); // place at the end
    BubbleUp(heap, heap.Count - 1); // restore heap property upward
}

```

ADVANCED — manual “bubble down” extraction, illustrating the mechanism behind Dequeue, including the subtlety of choosing the smaller child:

```

int ExtractMin(List<int> heap) {
    int min = heap[0];
    heap[0] = heap[^1]; // move last element to root
    heap.RemoveAt(heap.Count - 1);

    int index = 0;
    while (true) {
        int left = 2 * index + 1, right = 2 * index + 2;
        int smallest = index;

        if (left < heap.Count && heap[left] < heap[smallest]) smallest = left;
        if (right < heap.Count && heap[right] < heap[smallest]) smallest = right;
        if (smallest == index) break; // heap property satisfied

        (heap[index], heap[smallest]) = (heap[smallest], heap[index]);
        index = smallest;
    }
    return min;
}

```

The critical detail here: when bubbling down, you must compare against both children and swap with whichever is smaller (not just the left child arbitrarily) — swapping with the wrong child can leave the heap property violated relative to the other child, a subtle bug that's easy to introduce if this comparison is rushed.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, building a heap from an entire existing array (rather than inserting elements one at a time) can be done in $O(n)$ time using a technique called “heapify” — starting from the

last non-leaf node and bubbling down each node in reverse order — which is asymptotically faster than the naive $O(n \log n)$ you'd get from n individual $O(\log n)$ insertions, a non-obvious result worth knowing exists even without re-deriving the proof. There's also a broader family of heap variants beyond the basic binary heap shown here — Fibonacci heaps and binomial heaps offer better amortized complexity for certain operation patterns (especially decrease-key, used in advanced graph algorithms like Dijkstra's), though the standard binary heap covered here is what virtually all interview and practical contexts expect.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming a heap is fully sorted or behaves like a BST — the heap property only constrains parent-child relationships, not siblings or cross-subtree relationships, so assuming you can read off a sorted sequence by glancing at the heap's structure (the way 8.2's inorder traversal works on a BST) is incorrect; extracting elements one at a time via repeated ExtractMin/ExtractMax is the only way to get sorted output from a heap. Another is bubbling down by comparing against only the left child (or only checking one child) rather than both — this can leave the heap property violated against whichever child wasn't checked, a genuinely common implementation bug. A third: confusing a heap's array-index-based child/parent formulas — using $2i$ and $2i+1$ (a 1-indexed convention) when the array is actually 0-indexed (which needs $2i+1$ and $2i+2$) is an easy off-by-one mistake when implementing heap operations manually.

10. SELF-CHECK

- Why does the heap property guarantee $O(1)$ access to the minimum (or maximum), but not let you efficiently find the second-smallest element?
- Walk through, out loud, why “bubbling down” during extraction must compare against both children and choose the smaller (for a min-heap), rather than always swapping with the left child.
- Why is a heap typically implemented as an array rather than as an explicit tree with node objects and pointers, the way 8.1's tree was structured?

9.2 — PriorityQueue<TElement,TPriority> in C#

1. PLAIN-LANGUAGE GIST

C#'s built-in PriorityQueue<TElement,TPriority> is a ready-to-use min-heap that lets you enqueue items along with an explicit priority value, and always dequeues the item with the lowest priority value first — regardless of the order items were added, which is the defining difference from 4.2's plain Queue<T>.

2. REAL-WORLD ANALOGY

Picture a hospital emergency room's triage system: patients aren't seen in the order they walked in (that would be a plain queue) — they're seen based on severity, with the most critical case always treated next, even if they arrived last. PriorityQueue is exactly this triage system: arrival order is irrelevant; only the assigned priority determines who's “dequeued” (treated) next.

3. CONNECT TO PRIOR KNOWLEDGE

This is 9.1's min-heap mechanism, packaged as a ready-to-use C# collection — exactly the same relationship 7.1's built-in sorting has to 7.2/7.3's hand-implemented sorting algorithms: you understand the underlying mechanism (9.1), but in practice you reach for the built-in, well-tested implementation rather than hand-rolling your own heap array manipulation.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Enqueue(element, priority) adds an item along with a separate priority value. Dequeue() removes and returns the element with the lowest priority value (by default — PriorityQueue is a min-heap under the hood). Peek() looks at the lowest-priority element without removing it.

Next layer — the element/priority separation, and what it buys you:

- Unlike 9.1's raw heap examples (which stored a single comparable value), PriorityQueue<TElement,TPriority> explicitly separates what you're storing (the element, which can be any type, even one with no natural ordering) from how it's prioritized (a separate, simpler comparable value) — this separation means you don't need to make your stored type implement IComparable at all; you just need numbers (or any comparable type) for priority.
- **It's a min-heap by default:** lower priority numbers come out first. To simulate a max-heap behavior (highest priority first), the conventional trick is to negate the priority value when enqueueing (Enqueue(item, -priority)), or supply a custom comparer that reverses the comparison — there's no separate “MaxPriorityQueue” type in .NET, so this negation trick is worth knowing as the standard workaround.
- **Updating an item's priority after it's already enqueued isn't directly supported** — unlike some heap variants in other languages/libraries that support an explicit “decrease-key” operation, .NET's PriorityQueue has no built-in way to change an already-enqueued item's priority; the common workaround is to enqueue a new entry with the updated priority and treat older, stale entries as ignorable when dequeued (a “lazy deletion” pattern), which requires the consuming code to handle potential staleness explicitly.

5. VISUAL REPRESENTATION

PriorityQueue<TElement,TPriority>: element and priority are SEPARATE

Enqueue(element, priority) -- lower priority number = dequeued FIRST (min-heap by default)

"low fever", pri=2

"chest pain", pri=0

"sprained ankle", pri=3

Dequeue() returns "chest pain" FIRST -- lowest priority number wins, NOT order added

Key contrasts with Queue<T> (4.2):

Queue<T>: Dequeue always returns whatever was Enqueued FIRST (arrival order)

PriorityQueue: Dequeue returns whatever has the BEST priority, regardless of arrival order

Peek() and Dequeue() are $O(\log n)$; Enqueue() is $O(\log n)$ -- both costlier than Queue<T>'s $O(1)$, the price paid for maintaining priority order dynamically.

The diagram shows three enqueued items with different priorities: despite "chest pain" being enqueued second, its lower priority number (0, meaning more urgent) means it's dequeued first — directly illustrating the contrast with 4.2's plain queue, where arrival order alone would have determined dequeue order.

6. WHEN TO USE / WHEN NOT TO USE

Use PriorityQueue<TElement,TPriority> when:

- You need to repeatedly process items in order of some priority/urgency/value, with new items potentially arriving at any time — task schedulers, event simulation, “always handle the most important pending item next.”
- You're implementing 9.3's “k largest/smallest” pattern or a graph algorithm needing a priority-driven frontier (Dijkstra's shortest path, a Tier 2 topic that explicitly relies on this exact structure).

Wrong choice when:

- You need strict arrival-order (FIFO) processing — that's plain Queue<T> (4.2); using a priority queue with a manufactured “priority = arrival order” workaround is unnecessary complexity when Queue<T> already does this directly and more efficiently.
- You need to frequently update priorities of already-enqueued items and can't tolerate the lazy-deletion workaround's complexity — in that specific case, a different structure or a custom heap implementation with explicit decrease-key support might be necessary.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — basic enqueue/dequeue with separate element and priority, the exact scenario diagrammed above:

```
var triage = new PriorityQueue<string, int>();
triage.Enqueue("low fever", 2);
triage.Enqueue("chest pain", 0);    // lower number = more urgent
triage.Enqueue("sprained ankle", 3);

Console.WriteLine(triage.Dequeue()); // "chest pain" -- priority 0 wins
Console.WriteLine(triage.Dequeue()); // "low fever" -- priority 2 next
```

INTERMEDIATE — simulating a max-priority-first queue using the negation trick:

```
var maxPriority = new PriorityQueue<string, int>();
maxPriority.Enqueue("task A", -5); // negate: higher original priority...
maxPriority.Enqueue("task B", -9); // ...becomes a LOWER negated value...
maxPriority.Enqueue("task C", -2); // ...so it's dequeued FIRST

Console.WriteLine(maxPriority.Dequeue()); // "task B" -- originally 9
```

This is a standard, simple workaround rather than a separate API — negating priority values flips the min-heap's “lowest first” behavior into an effective “highest first” behavior, with no change to the underlying data structure at all.

ADVANCED — the lazy-deletion pattern for handling “updated” priorities, since direct priority updates aren't supported:

```
var pq = new PriorityQueue<string, int>();
var latestPriority = new Dictionary<string, int>(); // current "truth"

void UpdatePriority(string item, int newPriority) {
    latestPriority[item] = newPriority;
    pq.Enqueue(item, newPriority); // adds a NEW entry -- old still in pq
}

string DequeueValid() {
    while (pq.Count > 0) {
        pq.TryDequeue(out string item, out int priority);
        if (latestPriority.TryGetValue(item, out int current)
            && current == priority) {
            return item; // matches latest -- it's valid
        }
        // otherwise STALE entry from before an update -- skip it
    }
    return null;
}
```

This combines `PriorityQueue` with a `Dictionary` (2.2's pattern) tracking each item's current truth — every dequeue checks whether the popped entry's priority matches the latest known value for that item, silently discarding outdated entries that were never explicitly removed; this is a real, commonly-needed pattern precisely because of .NET's lack of a built-in decrease-key operation.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, `.NET's PriorityQueue<TElement, TPriority>` is implemented internally as a standard binary min-heap over an array (exactly 9.1's mechanism), so its `Enqueue/Dequeue` operations are $O(\log n)$, matching the complexity bounds derived from first principles in 9.1. There's also a real-world detail worth knowing: `PriorityQueue` does NOT guarantee stable ordering among elements with equal priority — if your use case needs ties broken by insertion order, you must explicitly encode that into the priority itself (e.g., using a tuple-like composite priority of `(priority, insertionSequenceNumber)`), since the underlying heap structure provides no such guarantee on its own.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming `PriorityQueue` supports updating an already-enqueued item's priority directly — it doesn't, and attempting to search for and “modify” an entry in place isn't supported by the API at all; the lazy-deletion workaround (or a custom heap implementation) is necessary when this capability is genuinely needed. Another is forgetting that `PriorityQueue` is a min-heap by default and being surprised when “high priority” items come out last — without the negation trick (or a custom comparer), higher priority numbers are treated as lower priority in the “process first” sense, an easy mix-up if “priority” is used colloquially to mean “importance” rather

than literally “the value used for min-heap ordering.” A third: assuming equal-priority items maintain insertion order — `PriorityQueue` provides no such stability guarantee, and code that silently relies on this behavior may work by coincidence in testing but isn't guaranteed to behave that way.

10. SELF-CHECK

- Why does `PriorityQueue<TElement,TPriority>` separate the element from its priority, rather than requiring the element type itself to implement comparison logic?
- Why does negating priority values when enqueueing simulate “highest priority first” behavior on a structure that's natively a min-heap?
- Why is the lazy-deletion pattern (tracking each item's latest priority separately, and discarding stale dequeued entries) necessary, given that .NET's `PriorityQueue` has no built-in decrease-key operation?

9.3 — Using a heap for “k largest/smallest” problems

1. PLAIN-LANGUAGE GIST

To efficiently find the k largest (or smallest) elements in a large collection — especially a stream too large to sort entirely — maintain a heap of size exactly k , using the counterintuitive trick of a min-heap to track the k largest (or a max-heap to track the k smallest): the heap's root is always the weakest member of your current “top k ,” letting you instantly decide whether each new element deserves a spot.

2. REAL-WORLD ANALOGY

Picture a talent scout who can only sponsor 3 athletes at a time, and wants to always be sponsoring the 3 fastest runners seen so far. Every time a new runner's time comes in, the scout only needs to know one thing: who's currently the slowest of their 3 sponsored athletes? If the new runner beats that slowest one, swap them in; otherwise, ignore the new runner entirely. The scout never needs to re-rank all 3 athletes from scratch — just compare against the current weakest link.

3. CONNECT TO PRIOR KNOWLEDGE

This is a direct, clever application of 9.1's heap mechanics combined with 9.2's PriorityQueue — and it connects to 7.4's lower-bound thinking in an interesting way: fully sorting the entire input to find the top k would cost $O(n \log n)$, but maintaining a size- k heap costs only $O(n \log k)$, a meaningfully better bound when k is much smaller than n , since you avoid ever needing to fully order the elements you ultimately discard.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: To find the k largest elements: maintain a min-heap capped at size k . For each new element: if the heap has fewer than k elements, just add it. Otherwise, compare the new element to the heap's root (the current minimum of your top- k set) — if the new element is larger, remove the root and insert the new element; otherwise, discard the new element entirely (it's not large enough to make the cut).

Next layer — why a min-heap is the right tool for finding the largest elements (the counterintuitive part):

- The heap's root, in a min-heap, is always the smallest of whatever's currently in the heap — when that heap is capped at size k and holding your current “top k ” candidates, the root is specifically the weakest member of that top- k set, i.e., exactly the one most likely to need replacing.
- This is why the heap type is “inverted” relative to what you're searching for: finding the LARGEST k elements uses a MIN-heap (so you can cheaply identify and evict the weakest current member), while finding the SMALLEST k elements uses a MAX-heap (so you can cheaply identify and evict the largest, weakest-for-that-purpose, current member).
- **Why this beats full sorting:** the heap never grows beyond size k , so every insertion/extraction is $O(\log k)$, not $O(\log n)$ — across all n elements in the input stream, the total cost is $O(n \log k)$, which for small, fixed k (even if n is huge or unbounded, as in a live data stream) is dramatically better than sorting everything first ($O(n \log n)$) just to take the top k afterward.
- This pattern works equally well on a continuous, unbounded stream of data (where you could never sort “everything” since there's no fixed endpoint) — a direct practical advantage over any sort-first approach, which fundamentally requires the full dataset to be available upfront.

5. VISUAL REPRESENTATION

Top-3 largest using a MIN-heap of size 3 (counterintuitive but correct)

Stream: 5, 1, 9, 3, 7, 2 -- keep only the 3 largest seen so far

```
see 5: heap=[5] (heap size < 3, just add)
see 1: heap=[1,5] (heap size < 3, just add)
see 9: heap=[1,5,9] (heap size < 3, just add)
see 3: 3 > heap min (1)? yes -- pop 1, push 3 -- heap=[3,5,9]
see 7: 7 > heap min (3)? yes -- pop 3, push 7 -- heap=[5,7,9]
see 2: 2 > heap min (5)? NO -- skip entirely, heap unchanged
```

Final heap: {5, 7, 9} -- exactly the 3 largest values seen, in any order

The heap's MINIMUM is always the weakest of your current "top k" --
comparing new elements against it tells you instantly whether they deserve a spot.

**Heap never exceeds size k -- each operation is $O(\log k)$, not $O(\log n)$ --
a real win when k is small and n (the full stream) is huge.**

The diagram traces finding the top-3 largest values from the stream 5, 1, 9, 3, 7, 2: the heap fills up to size 3 first, then each subsequent element is compared only against the heap's current minimum — 3 and 7 both beat the heap's weakest member and get swapped in, while 2 doesn't beat anything and is discarded without ever touching the heap.

6. WHEN TO USE / WHEN NOT TO USE

Use the size-k heap pattern when:

- You need the k largest/smallest elements (or the k-th largest/smallest specifically) from a large or streaming collection, and k is meaningfully smaller than the total input size n.
- The data arrives incrementally (a live stream, or data too large to hold and sort all at once) — the heap approach naturally handles this one element at a time, without needing the full dataset available upfront.

Wrong choice when:

- You need all elements in fully sorted order, not just the top k — at that point, sorting (7.2/7.3) is the more direct tool, since maintaining a size-k heap doesn't help if k equals (or is close to) n.
- k is very close to n — the asymptotic advantage ($O(n \log k)$ vs $O(n \log n)$) shrinks to nothing as k approaches n, and the added complexity of heap management isn't worth it for a negligible gain.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — top-k largest elements, the exact trace above, using PriorityQueue as the min-heap:

```

List<int> TopKLargest(int[] nums, int k) {
    var minHeap = new PriorityQueue<int, int>(); // priority = value

    foreach (int num in nums) {
        if (minHeap.Count < k) {
            minHeap.Enqueue(num, num);
        } else if (num > minHeap.Peek()) {
            minHeap.Dequeue(); // evict weakest
            minHeap.Enqueue(num, num);
        }
        // else: num isn't large enough to make the cut -- discard
    }

    var result = new List<int>();
    while (minHeap.Count > 0) result.Add(minHeap.Dequeue());
    return result;
}

```

INTERMEDIATE — k-th largest element specifically (a common, slightly different framing of the same pattern):

```

int KthLargest(int[] nums, int k) {
    var minHeap = new PriorityQueue<int, int>();

    foreach (int num in nums) {
        minHeap.Enqueue(num, num);
        if (minHeap.Count > k) {
            minHeap.Dequeue(); // keep heap size capped at exactly k
        }
    }
    return minHeap.Peek(); // root IS the k-th largest, by construction
}

```

After processing all elements, the heap holds exactly the k largest values, and since it's a min-heap, its root is specifically the smallest of those k — which is, by definition, the k-th largest overall. This is a subtly different (and often simpler) framing than tracking “the top k as a set” — you just need the one boundary value.

ADVANCED — top-k frequent elements, combining 2.3's frequency counting with this topic's heap pattern:

```

List<int> TopKFrequent(int[] nums, int k) {
    var freq = new Dictionary<int, int>(); // 2.3's frequency map
    foreach (int num in nums)
        freq[num] = freq.GetValueOrDefault(num, 0) + 1;

    var minHeap = new PriorityQueue<int, int>(); // priority = frequency
    foreach (var pair in freq) {
        if (minHeap.Count < k) {
            minHeap.Enqueue(pair.Key, pair.Value);
        } else if (pair.Value > minHeap.Peek()) {
            // Peek() only returns the ELEMENT, not priority -- need
            // TryPeek with priority, or track priorities separately
        }
    }
    // (Simplified illustration -- a full implementation needs
    // TryPeek(out element, out priority) to compare against the
    // heap's current minimum FREQUENCY, not just re-derive it)
    return new List<int>(); // placeholder for illustration
}

```

This demonstrates the natural two-step combination: first build a frequency map (2.3), then apply this topic's exact size-k-heap pattern — but using frequency as the priority instead of the raw

value, showing that “top k by some criterion” generalizes cleanly to any computable priority, not just an element's own value.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, there's an alternative approach to “k-th largest” specifically (not the full top-k set, just the boundary value) using a randomized variant of 7.3's quickselect (a close cousin of quicksort's partitioning, but recursing into only one side rather than both) — this achieves expected $O(n)$ time, better than the heap approach's $O(n \log k)$, though it requires the full dataset upfront and doesn't handle streaming data, a direct tradeoff against the heap method's incremental, streaming-friendly nature. There's also a connection worth flagging: this exact size-k-heap pattern is the standard building block behind “k nearest neighbors” style problems (finding the k closest points to a target), where “priority” becomes distance rather than raw value — same heap mechanism, different priority computation.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is using a max-heap instead of a min-heap when looking for the k largest elements — this inverts the entire mechanism, since a max-heap's root would be the current largest found so far, giving you no easy way to identify and evict the weakest member of your top-k set, which is the actual operation this pattern depends on. Another is forgetting to check the heap's current size before deciding whether to simply add a new element or compare-and-possibly-replace — skipping the “if heap size < k” branch and always comparing against `Peek()` would either crash (peeking an empty heap) or behave incorrectly until the heap reaches size k. A third: applying this pattern when k is close to n, missing that the complexity advantage ($O(n \log k)$ vs $O(n \log n)$) becomes negligible, and the simpler, more readable “just sort everything and take the top k” approach would be preferable in that specific regime.

10. SELF-CHECK

- Why does finding the k largest elements require a min-heap, rather than the perhaps more intuitive choice of a max-heap?
- In the “k-th largest” example, why does the heap's root specifically correspond to the k-th largest value once the heap has processed the entire input and settled at size k?
- Why does this heap-based approach handle a continuous, unbounded data stream more naturally than a “sort everything, then take the top k” approach?

9.4 — Using two heaps for median-tracking problems (concept)

1. PLAIN-LANGUAGE GIST

To efficiently track the running median of a continuously growing stream of numbers, maintain two heaps that split the data in half — a max-heap holding the smaller half of the numbers seen so far, and a min-heap holding the larger half — keeping them balanced in size so that the median is always immediately available at the top of one (or both) of the heaps, without ever needing to fully sort the data.

2. REAL-WORLD ANALOGY

Picture organizing a line of people by height into two groups: everyone shorter than the midpoint stands in one group (ordered so the tallest of this “shorter” group is easiest to find), and everyone taller stands in another group (ordered so the shortest of this “taller” group is easiest to find). The median height is always right at the boundary between these two groups — specifically, it's whichever of “the tallest of the shorter group” or “the shortest of the taller group” you need, depending on whether the total count is odd or even.

3. CONNECT TO PRIOR KNOWLEDGE

This is a direct, elegant combination of 9.1's min-heap and max-heap mechanics used together, deliberately exploiting the fact that a max-heap gives $O(1)$ access to “the largest of this subset” while a min-heap gives $O(1)$ access to “the smallest of this subset” — exactly the two pieces of information needed to identify a median sitting at the boundary between two halves.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Maintain a max-heap for the lower half of all numbers seen so far, and a min-heap for the upper half. For each new number: decide which half it belongs in (compare it to the max-heap's current root), insert it into the appropriate heap, then rebalance if the two heaps' sizes differ by more than one element (by moving the root of the larger heap into the other heap).

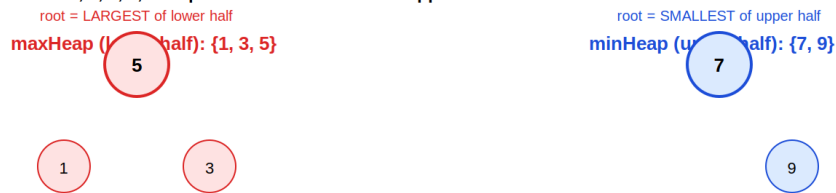
Next layer — why this specific arrangement makes the median trivially accessible:

- The max-heap's root is the LARGEST element of the lower half — conceptually, “the highest of the low scores.”
- The min-heap's root is the SMALLEST element of the upper half — “the lowest of the high scores.”
- If the two heaps are kept balanced in size (differing by at most one element), these two roots are precisely the two values immediately surrounding the true median position in the fully sorted data, without ever actually sorting it.
- **If the total count is odd**, one heap has exactly one more element than the other — that heap's root IS the median directly.
- **If the total count is even**, both heaps have equal size, and the median is the AVERAGE of the two roots (the largest of the lower half and the smallest of the upper half).
- **The rebalancing step is essential and easy to get subtly wrong:** every insertion must be followed by a check — if one heap has grown to have 2 or more elements than the other, move its root into the other heap, restoring the size balance. Skipping this check, or rebalancing by moving the wrong element, breaks the entire “roots are adjacent to the median” guarantee that this whole technique depends on.

5. VISUAL REPRESENTATION

Two heaps split the data stream in half, balanced around the median

Stream so far: 5, 1, 9, 3, 7 -- split into "lower half" and "upper half"



maxHeap has 3 elements, minHeap has 2 -- maxHeap's root (5) IS the median
(odd total count -- median is the extra element's heap's root)

Every new number goes into maxHeap (lower half) or minHeap (upper half)
based on comparison to maxHeap's current root, then heaps are REBALANCED
so their sizes never differ by more than 1.

Median is always just the root(s) -- $O(1)$ to read, $O(\log n)$ to insert a new number.

The diagram shows the stream 5, 1, 9, 3, 7 split into a max-heap holding the lower half {1, 3, 5} and a min-heap holding the upper half {7, 9} — with five total elements (odd), the max-heap (holding the extra element) has its root, 5, sitting exactly at the median position.

6. WHEN TO USE / WHEN NOT TO USE

Use the two-heap median pattern when:

- You need to repeatedly query the median of a data stream as it grows, without re-sorting the entire dataset from scratch on every query — “find median from data stream” is the canonical, frequently-asked version of this exact problem.
- You need other order-statistics-style queries that benefit from maintaining a balanced split around a moving boundary, even if the specific value needed isn't always the exact midpoint.

Wrong choice when:

- You only need the median once, on a static, unchanging dataset — sorting the data once (7.2/7.3) and directly indexing the middle element(s) is simpler and equally efficient for a single query, without the ongoing bookkeeping overhead of maintaining two balanced heaps.
- You need percentiles or order statistics other than the median specifically (e.g., the 90th percentile) on a frequently-updated stream — the two-heap technique is specifically calibrated around the 50/50 split; other percentiles would need a differently-proportioned splitting strategy or a different structure entirely.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — the basic two-heap structure and insertion logic, the exact scenario diagrammed above:

```

class MedianFinder {
    private PriorityQueue<int, int> maxHeap = new(); // lower half
    private PriorityQueue<int, int> minHeap = new(); // upper half

    public void AddNumber(int num) {
        if (maxHeap.Count == 0 || num <= -maxHeap.Peek()) {
            maxHeap.Enqueue(num, -num); // negate for max-heap behavior
        } else {
            minHeap.Enqueue(num, num);
        }
        Rebalance();
    }

    private void Rebalance() {
        if (maxHeap.Count > minHeap.Count + 1) {
            int moved = -maxHeap.Dequeue(); // un-negate
            minHeap.Enqueue(moved, moved);
        } else if (minHeap.Count > maxHeap.Count + 1) {
            int moved = minHeap.Dequeue();
            maxHeap.Enqueue(moved, -moved);
        }
    }
}

```

This directly reuses 9.2's negation trick to simulate a max-heap using .NET's natively min-heap `PriorityQueue` — every value pushed into `maxHeap` is stored with its negated priority, and every value read back out (via `Dequeue` or `Peek`) must be re-negated to recover the original value.

INTERMEDIATE — retrieving the median, handling both the odd-count and even-count cases:

```

public double FindMedian() {
    if (maxHeap.Count > minHeap.Count) {
        return -maxHeap.Peek(); // odd total -- maxHeap has the extra
    }
    if (minHeap.Count > maxHeap.Count) {
        return minHeap.Peek(); // odd total -- minHeap has the extra
    }
    // even total -- average the two boundary roots
    return (-maxHeap.Peek() + minHeap.Peek()) / 2.0;
}

```

This directly implements the odd/even distinction from section 4 — checking which heap (if either) has the extra element, and falling back to averaging both roots only when the counts are exactly equal.

ADVANCED — a worked trace through a sequence of insertions, explicitly showing the rebalancing step firing:


```
// Trace: AddNumber(5), AddNumber(1), AddNumber(9), AddNumber(3), AddNumber(7)
//
// Add 5: maxHeap empty -> maxHeap.maxHeap={5}, minHeap={}
// Add 1: 1 <= 5 -> maxHeap.maxHeap={5,1}, minHeap={}
//       Rebalance: maxHeap(2) > minHeap(0)+1 -> move root (5)
//       maxHeap={1}, minHeap={5}
// Add 9: 9 > 1 -> minHeap.maxHeap={1}, minHeap={5,9}
//       Rebalance: minHeap(2) > maxHeap(1)+1? No -- no move
// Add 3: 3 > 1 -> minHeap.maxHeap={1}, minHeap={5,9,3}
//       Rebalance: minHeap(3) > maxHeap(1)+1 -> move root (3)
//       maxHeap={1,3}, minHeap={5,9}
// Add 7: 7 > 3 -> minHeap.maxHeap={1,3}, minHeap={5,9,7}
//       Rebalance: minHeap(3) > maxHeap(2)+1? No -- no move
//
// Final: maxHeap={1,3} (root 3), minHeap={5,7,9} (root 5)
// minHeap has the extra element -> median = 5
```

Hand-tracing this sequence (0.8's skill, directly applied) is genuinely worth doing once — it makes the abstract “rebalance when sizes differ by more than 1” rule concrete, and shows exactly how elements migrate between heaps as the rebalancing step fires only when truly needed.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, the two-heap median technique generalizes into maintaining arbitrary order statistics (not just the median) by adjusting the target split ratio — e.g., maintaining the 25th percentile would mean keeping the “lower” heap at roughly a quarter of the total size rather than half — though this generalization requires more careful rebalancing logic than the simple 50/50 median case shown here. There's also a connection worth noting to order-statistics trees (briefly mentioned in 8.4's deeper knowledge) — a self-balancing BST augmented with subtree-size metadata achieves similar “find the k-th smallest element dynamically” capabilities, and is a viable alternative to the two-heap approach for problems needing more flexible order-statistics queries than just the median specifically.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is forgetting the rebalancing step after insertion, or rebalancing inconsistently — without strictly maintaining the “sizes differ by at most one” invariant after every single insertion, the two heap roots no longer reliably sit at the boundary positions the median calculation depends on, silently producing wrong median values. Another is mixing up which heap is which — using a min-heap for the lower half (where you'd want fast access to the largest of that half) or a max-heap for the upper half (where you'd want fast access to the smallest) inverts the entire mechanism and makes the roots meaningless for median purposes. A third: forgetting to un-negate values when reading from a max-heap simulated via negated priorities (the C# PriorityQueue workaround from 9.2) — returning the raw negated priority value instead of converting it back produces a median that's numerically wrong by a sign flip, an easy oversight when combining 9.2's trick with this topic's two-heap structure.

10. SELF-CHECK

- Why does keeping the two heaps' sizes balanced (differing by at most one) matter for the correctness of the median calculation — what specifically would go wrong if one heap were allowed to grow much larger than the other?
- Walk through, out loud, why the max-heap's root represents “the largest of the lower half” and why that's exactly the right piece of information needed when the total count is odd and the max-heap holds the extra element.

- Why is a max-heap used for the lower half and a min-heap for the upper half, rather than the reverse — what specific property of each heap type makes this pairing the correct one?